CSE 445: Introduction to Machine Learning

Assignment 1 — Classification with Perceptrons across Datasets

Due: October 22 2025
In-class Kaggle Competition Date: October 23 2025

Overview

Build and compare perceptron models on the Iris dataset: implement your own Single-Layer Perceptron (SLP) and a one-hidden-layer Multi-Layer Perceptron (MLP) from scratch, then benchmark against scikit-learn implementations with automated feature and hyperparameter selection.
Following this, there will be an in-class Kaggle competition, where the same from-scratch MLP implementation will be applied to a different dataset for a classification task.

Setup

- **Dataset:** `sklearn.datasets.load_iris` only
- **Environment:** Python 3.x with NumPy, Pandas, Matplotlib, scikit-learn
- **Important:** Use the same train/test split and random seed across all models for fair comparison.

Tasks

Part 1: Data Exploration

Perform brief Exploratory Data Analysis (EDA) including class distribution, feature statistics, and 2–3 **meaningful** visualizations that inform your feature selection.

Part 2: From-Scratch Implementation

Build your own SLP and MLP using NumPy (no sklearn models here):

- Select **two features** based on your EDA and justify your choice. **Hint:** You may want to look at the correlation analysis of the features.
- Implement a SLP for multi-class classification using One-vs-Rest (OvR) strategy with perceptron learning rule.
- Implement a **one-hidden-layer** MLP with `tanh` or `ReLU`, train with `Stochastic Gradient Descent (SGD)` - momentum optional. **Do not** use Adam.
- Plot decision boundaries at **every 5th epoch** for both models during training.
- Evaluate on the test set with a **classification report** and **confusion matrix**.

Part 3: Library Baselines

Use scikit-learn's `Perceptron` and `MLPClassifier`:

- Use a **Pipeline** with scaling and `SelectKBest`, wrap the pipeline in `GridSearchCV`. The feature selector **must be inside** the pipeline/CV (no fitting on the full data).
- Include `num_features` as a hyperparameter ($k \in \{2, 3, 4\}$) alongside model parameters.
- For MLP, use `sgd` solver, do not use Adam.
- Report test performance (**classification report** and **confusion matrix**), best parameters (especially the optimal `num_features`), and which features were selected.

Part 4: Report

Write a maximum of 3-page report (IEEE Conference Template) covering:

- Implementation approach and feature selection rationale
- Key visualizations and results for all models
- Comparison between from-scratch and library implementations
- Citations and Resources:
  - Any direct references to papers, textbooks, or code repositories must be listed under a **References** section **in the report** (IEEE citation format). PS: For this assignment, literature review is not required.
  - Background sources (e.g., tutorials, StackOverflow) and AI tools (e.g., ChatGPT, Copilot) should be noted in a short **Resources** section at the **end of the notebook**.

Part 5: In-class Kaggle Competition

A Kaggle competition will be held during class time (link and details will be shared on the competition day). Attendance in class is mandatory: submissions made without being present will not be considered unless an official excuse is provided.

No prior information about the competition dataset will be given. During the session, EDA, feature engineering, and classification with the from-scratch MLP implementation are expected.

For preparation, it is recommended to review general approaches to EDA and feature engineering. A useful reference is the following tutorial: `https://www.kaggle.com/code/gunesevitan/titanic-advanced-feature-engineering-tutorial`

Submission

Submission Instructions

Platform: All submissions must be made through **Moodle** by the **designated group submitter only**.

Submission Guidelines (Do not compress into .zip or .rar):

- *groupID*`_assignment1.ipynb` — Must execute successfully with *Restart & Run All*.
- *groupID*`_assignment1.pdf` — Report prepared in the IEEE Conference Template format.
- `requirements.txt` — List of additional packages used (if any).
- `README.md` — Brief setup or usage instructions (if applicable).

Grading

Component	Weight
Notebook + Report	50%
Kaggle Competition	50%

Special Rule: From-scratch MLP class implementations will be shared without modification in the Kaggle competition. Only parameter **values** may be changed. Adding/removing parameters or adding/removing/updating functions will result in a deduction of points.

Hint: Make sure your model can handle more than 2 features and more/less than 3 classes.

Automatic rules (applied before scoring):

- **Valid Experiment Conditions breach** — any of: data leakage, inconsistent splits across models (not using same train and test sets across models): **Penalty: −10 each, −20 max).**
- **Not** using the **same** from-scratch MLP models across *groupID*`_assignment1.ipynb` notebook and Kaggle competition: **Penalty: −10**
- Missing seeded randomness: **Penalty: −5**

Bonus (max +25, final grade capped at 100):

- **Hyperparameter Tuning (+0–10):** Systematically test at least three different learning rates for your from-scratch MLP. Report the final test accuracy for each and identify the best-performing one.
- **Model Interpretability (+0–15):** For the `scikit-learn` `MLPClassifier`, use **permutation importance** (`sklearn.inspection.permutation_importance`) to find and visualize feature importances on the test set. Then, implement a simplified version of permutation importance from scratch for **your own MLP**. Compare the feature rankings with the scikit-learn version's results.

Academic Integrity

Zero Grade Conditions:

- Cannot reproduce with *Restart & Run All*
- Notebook does not reproduce reported results
- Code does not match the described methodology
- Similarity detection (for This Assignment): ≥60% (–50% grade), near-identical (zero grade)
- **Note:** All group members are equally responsible for plagiarism violations

Environment & Runtime Policy

All submissions must execute on **CPU only** (no GPU/TPU). The complete notebook run ("Restart & Run All") must finish within **10 minutes**. Submissions exceeding this limit will incur a **−10 points** penalty.